INNOPOLIS UNIVERSITY

SNE MASTER TRACK

2025

COURSE: [F25] COMPUTER SYSTEMS AND NETWORKS

CSN LAB 5 REPORT

On

Linux daemon configuration

Student Name: HASIB AL TAHSIN

Application ID: 187623

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 08.10.2025

Task 1: Theory

What is systemctl and init and systemd ?

what are the available Runlevel on linux?

what does the systemctl list-unit-files command does?

Solution:

1. systemctl, init ও system

init (Initialization) is the first process (often PID = 1) to start the operating system after the kernel, which starts the associated services and daemons. It acts as a manager that executes scripts and tasks at various runlevels [1], [3].

systemd is a modern init and system management system that overcomes the old limitations of the init system. systemd is widely used in Linux distributions. It is flexible, fast, and convenient, providing features such as service dependency management, seamless startup, configuration via unit files, etc. [4].

systemctl is a CLI (Command Line Interface) command tool used to control and manage systemd. Using systemctl, you can start/stop services, view status, load/unload unit files, etc. [3].

2. Available Runlevels in Linux

Linux / Unix uses the concept of runlevels, which are different operating "modes" in which various services are enabled or disabled. Each runlevel represents a specific state [4]. It is generally numbered 0–6:

Runlevel	Work
0	System halt
1	single-user mode (maintenance)
2	multi-user (without GUI & network) (varies on distribution)
3	multi-user mode + text mode
4	User defined (distribution dependent)
5	multi-user + Graphical interface (GUI)

Some distributions use runlevel 5 as the default GUI mode [4], [5]. There are some targets (such as graphical.target, multi-user.target) that are equivalent to runlevels [3].

3. What the systemctl list-unit-files command does

The systemctl list-unit-files command is a command used in the Linux systemd environment that displays all unit files and their status. It tells you which unit files are enabled, which are disabled, which are static, etc. [2].

The specified output shows:

• UNIT FILE - Unit file name (e.g. sshd.service, cron.service)

• STATE - Its state e.g. enabled, disabled, static, masked etc.

This explains the system's default service management.

Task 2: Creating Systemd service

Create a shell script that would write a text to a file echo "bla bla" > /tmp/test.txt and then create a Systemd service that would this script. Here are the requirements:

1. It should run as your current user

2. the working directory should be set to the current user home directory

3. this service will only run after the network.target service

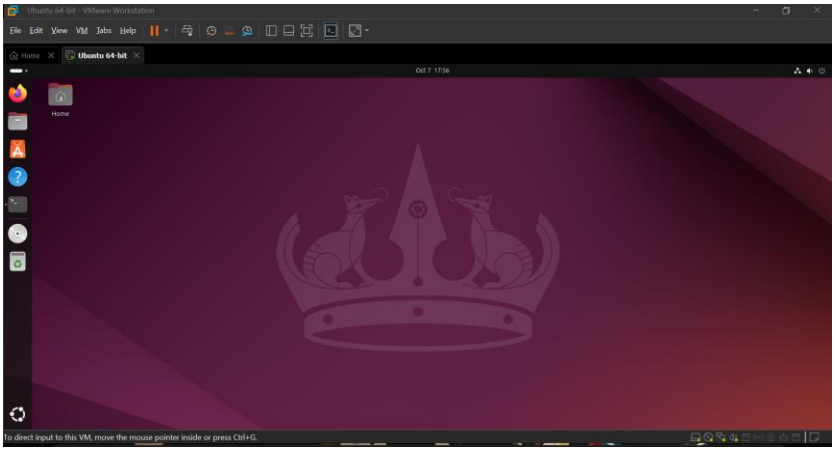
Solution:

Creating a shell script

The goal is to create a shell script that writes "bla bla" to the file /tmp/test.txt.

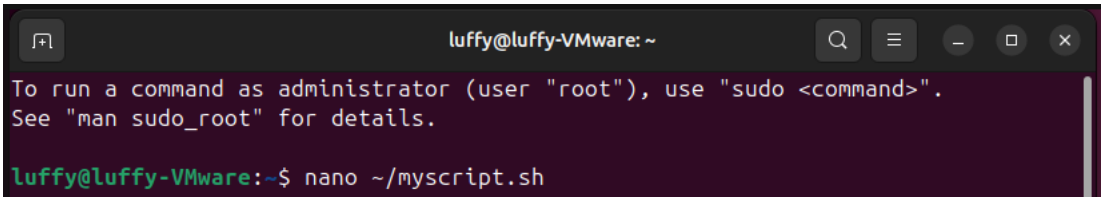
Starting Ubuntu

I opened VMware on the Windows desktop and started the Ubuntu VM and logged in when Ubuntu started.



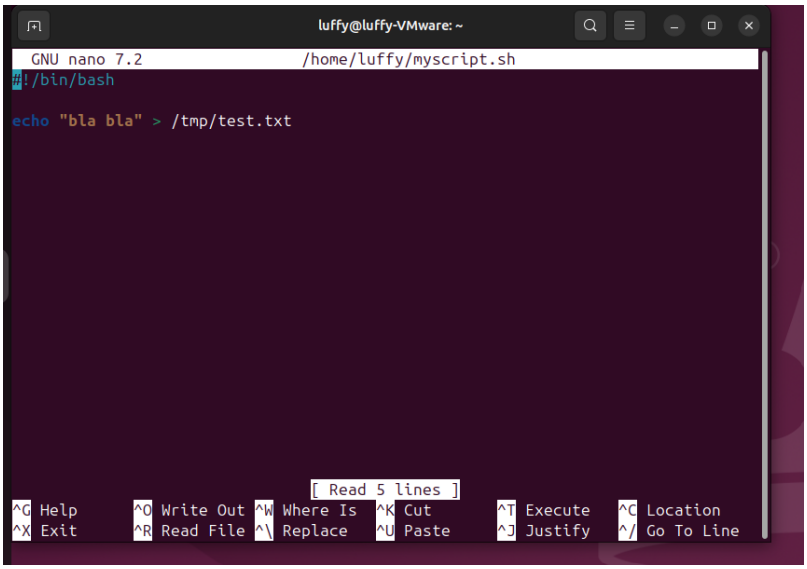
Creating a script file

After typing the following command in the terminal:



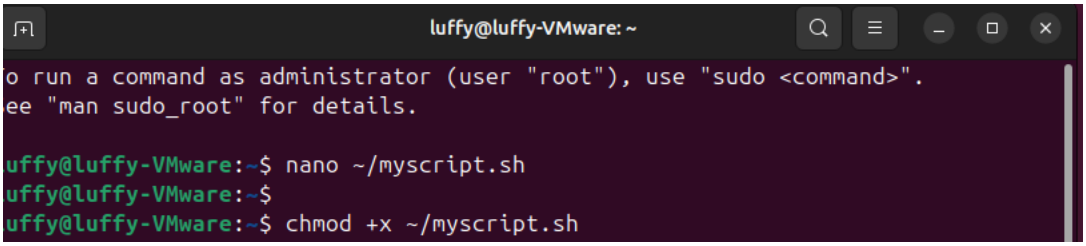
This opens the nano editor and creates a file called myscript.sh in the home directory [6], [7].

I Typed the following lines inside the editor and saved the file by pressing Ctrl + O on the keyboard. Then I pressed Enter and pressed Ctrl + X to exit nano.



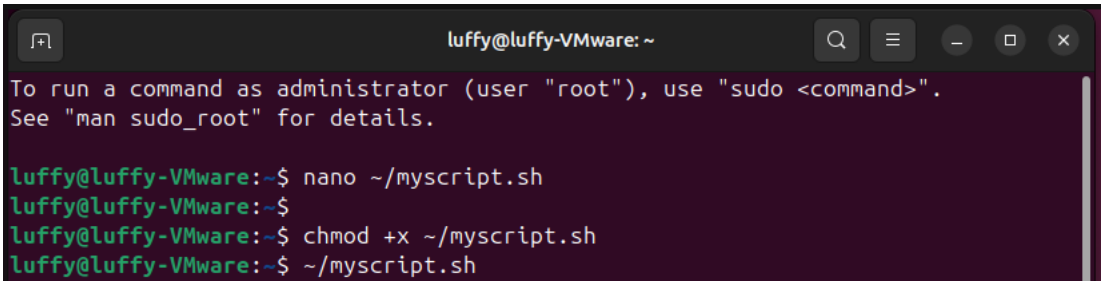
Granting execution permissions to the script

The following command makes the script executable.

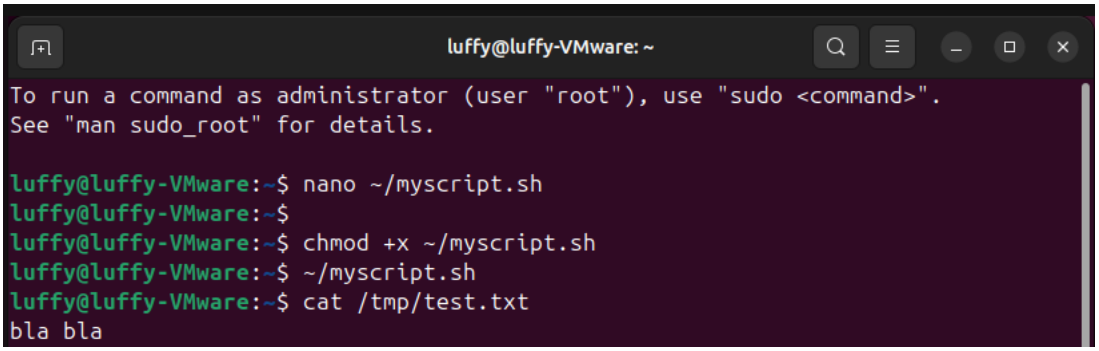


Testing the script

The following command is used to check whether the script is working or not:



Then checking whether the file was created or not:



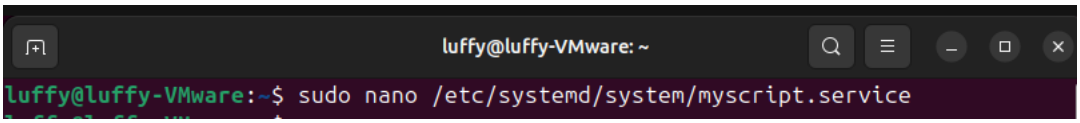
The output shows bla bla.

Creating a systemd service

The goal is to create a systemd service that:

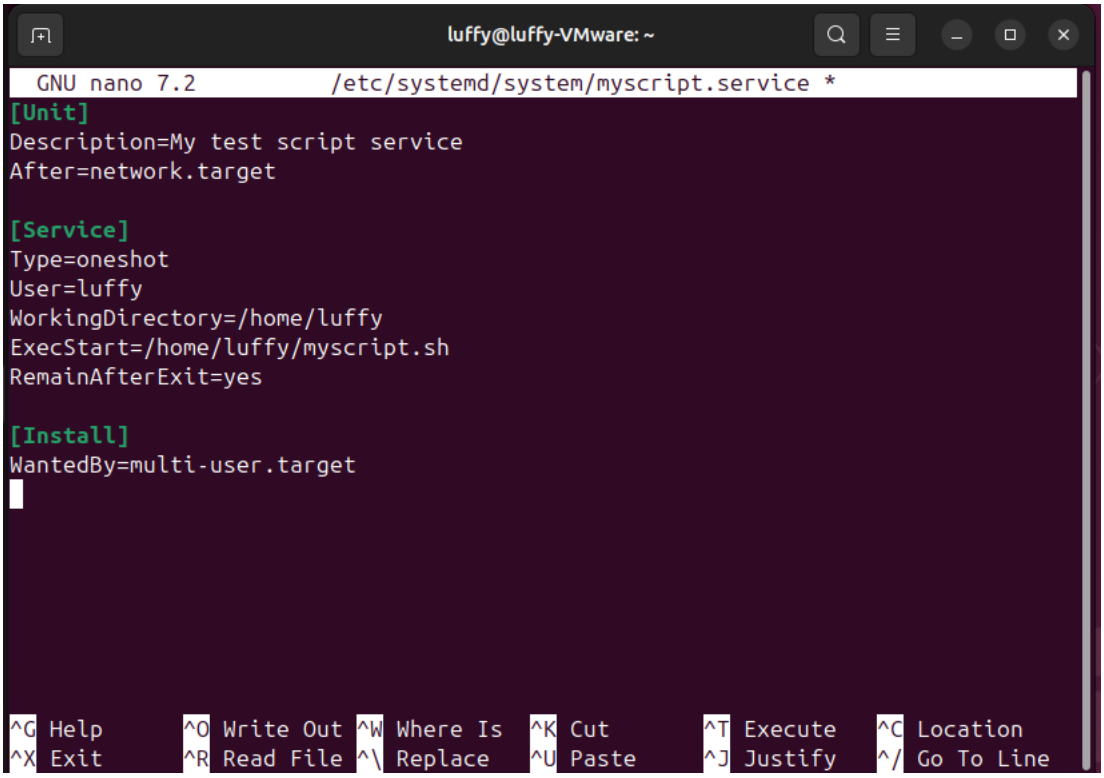
- Will run as the current user
- The working directory will be the user's home directory
- Will be started after network.target [6], [7]

To create a new service file, I typed in the terminal:



This creates a systemd service file.

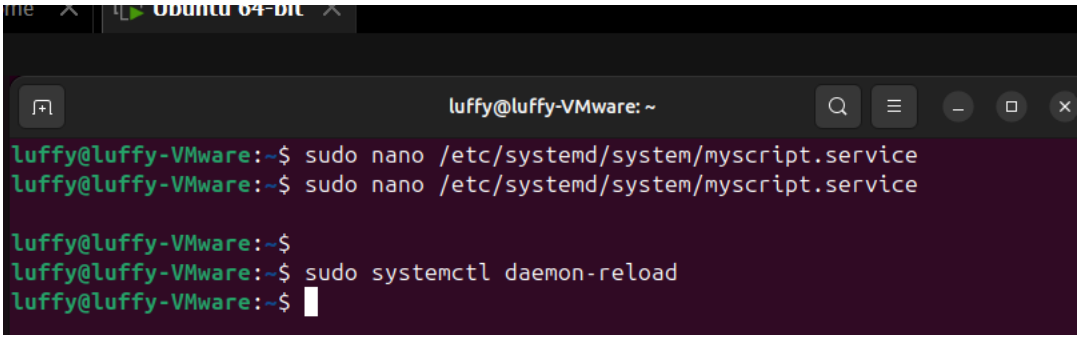
Writing the following content using editor and saving it:



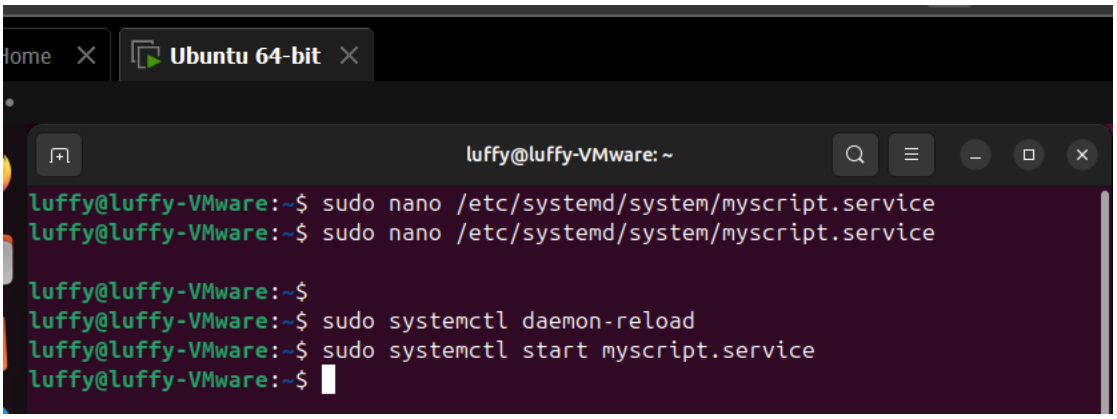
In the <username> section, my real Ubuntu username Luffy was inserted. Then the content was saved via Ctrl + O → Enter → Ctrl + X.

Reload systemd

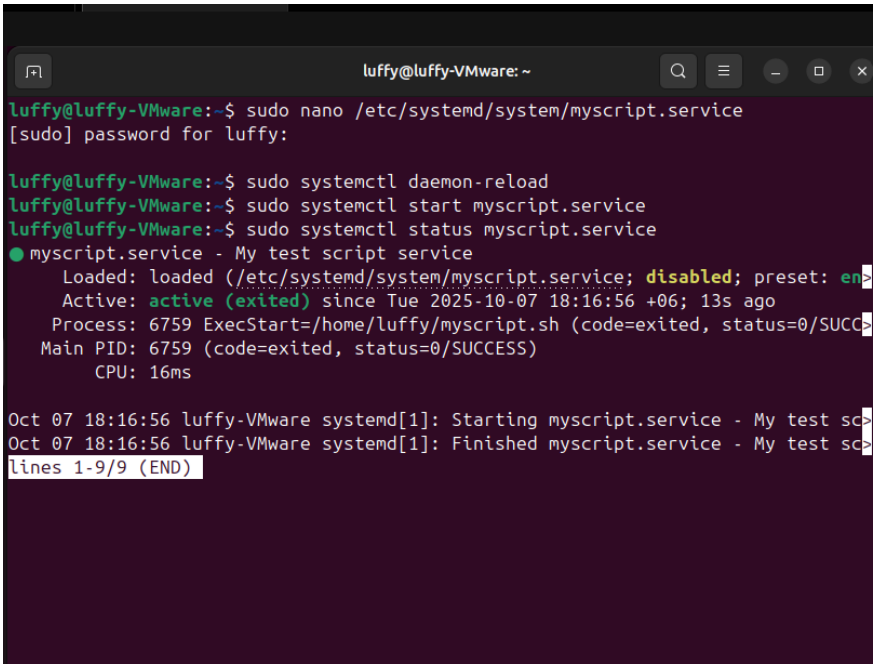
Reloading systemd to register the service:



Starting the service



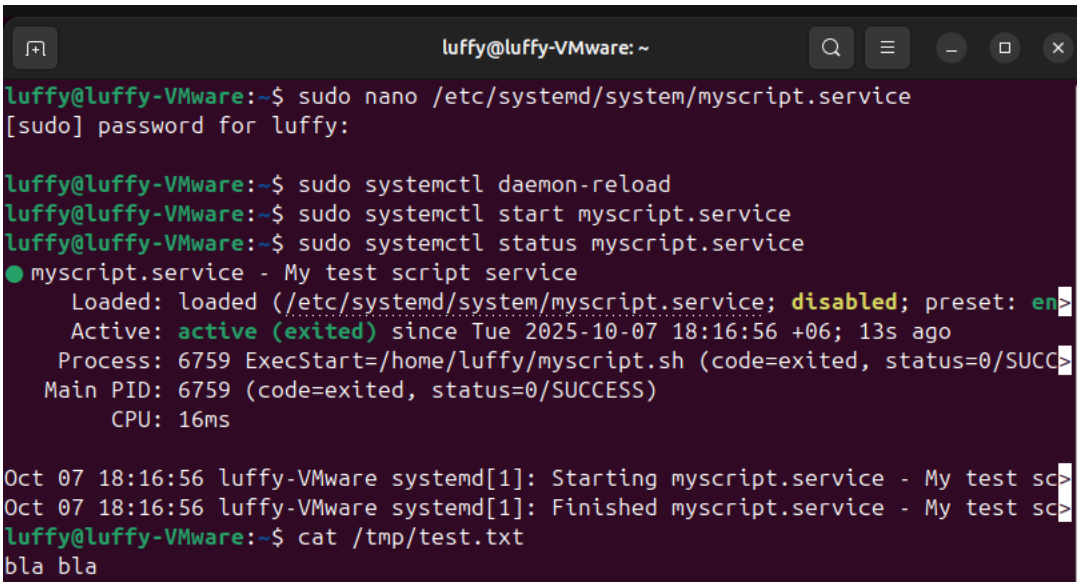
Checked whether the service is running or not.



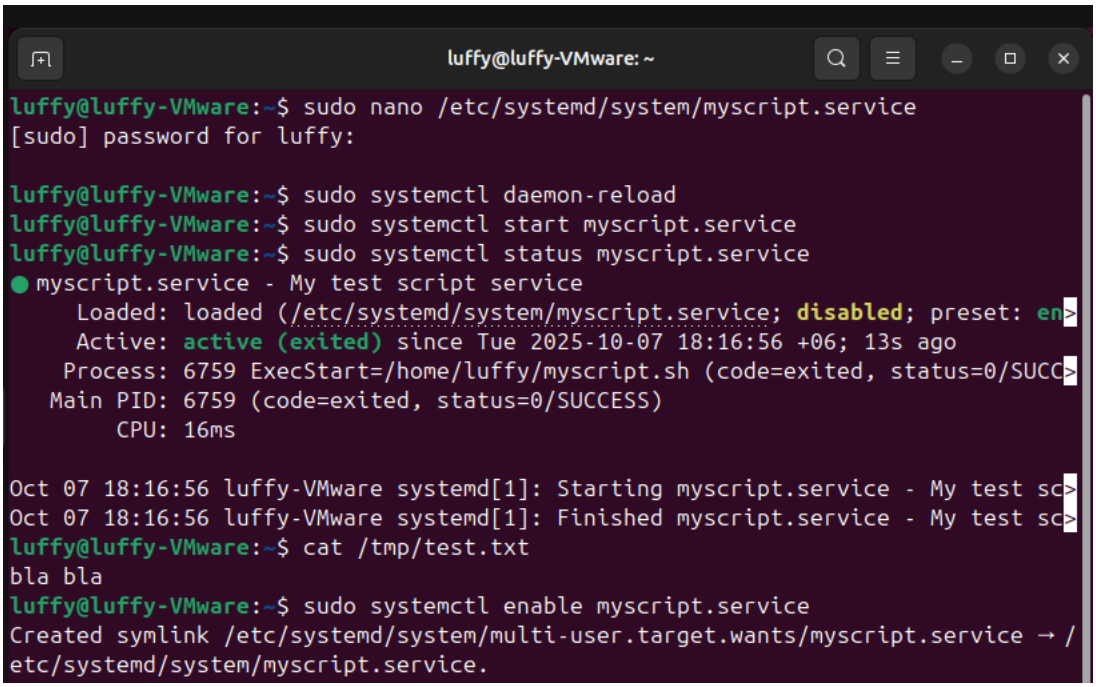
Active Status: Active: active (running) is seen in the terminal.

Testing

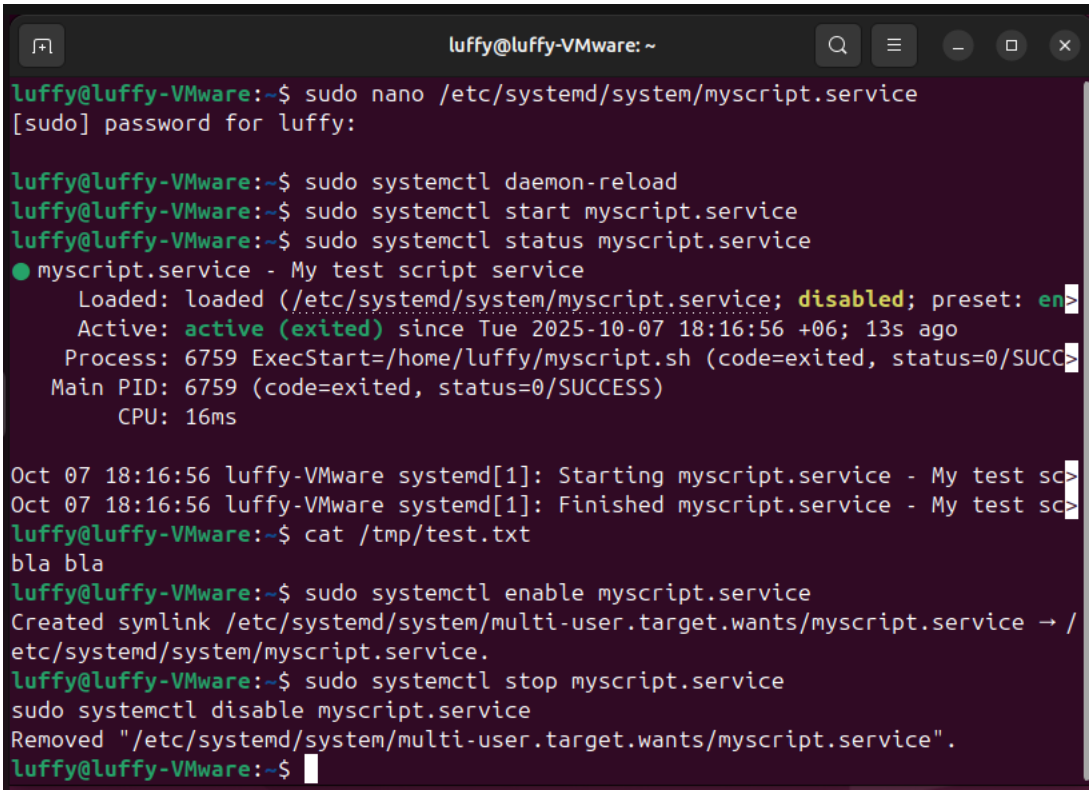
Now we check that the file /tmp/test.txt has been created. The result shows bla bla.



To enable the service at startup so that it runs automatically when Ubuntu starts [8]:



Stopping or disabling the service



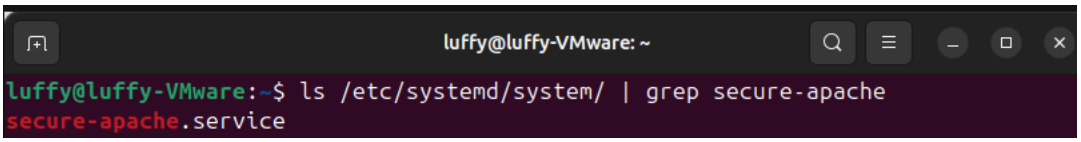
Task 3: Install a web-server service

install a web-server (apache, nginx, httpd) on a virtual machine and create a Systemd service for that web-server. Please make sure to write a secure service.

Solution:

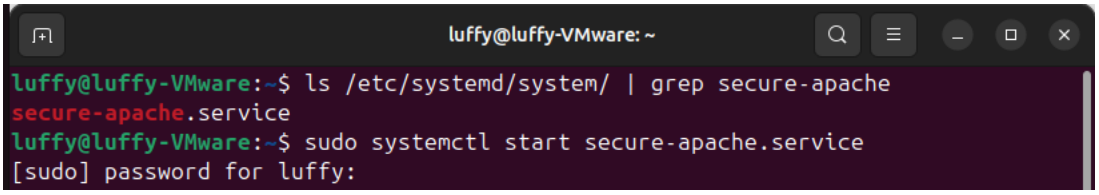
Checking the service file

The service name is secure-apache.service, then I have checked if it exists:



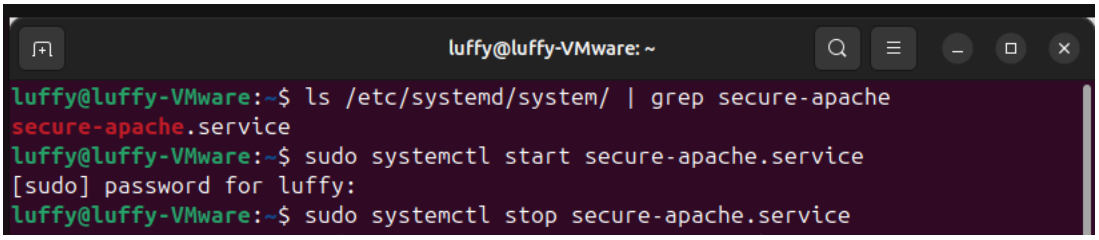
Starting the service

The following command starts the Apache server [9], [10].



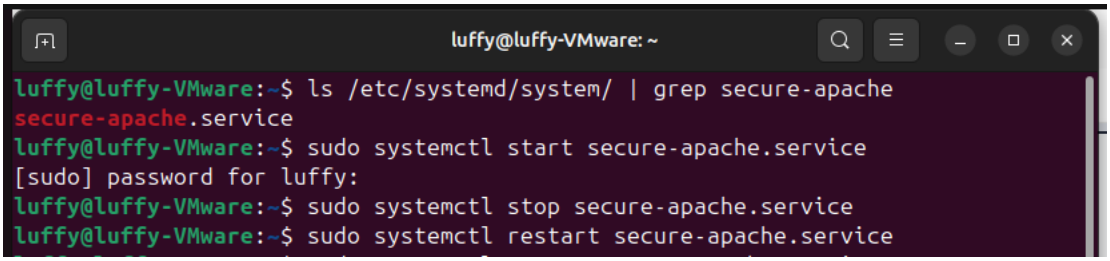
Stopping the service

The service is stopped by the following command [9], [10].



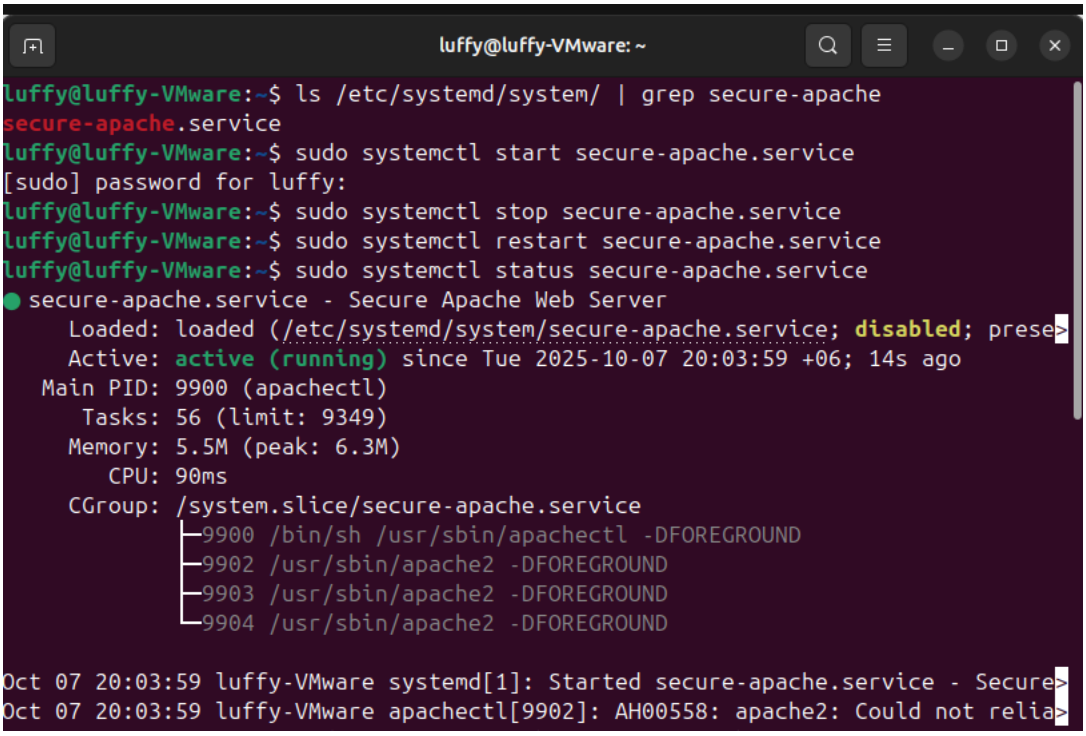
Restarting the service

The service is restarted after the configuration change [10].



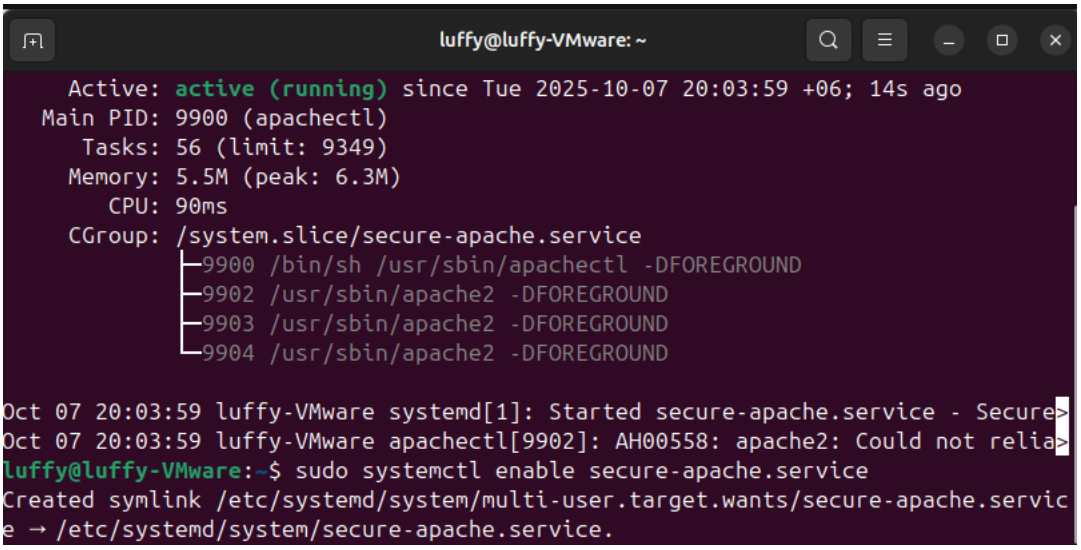
Viewing the service status

At this stage, the Service Status is showing Active (Running).



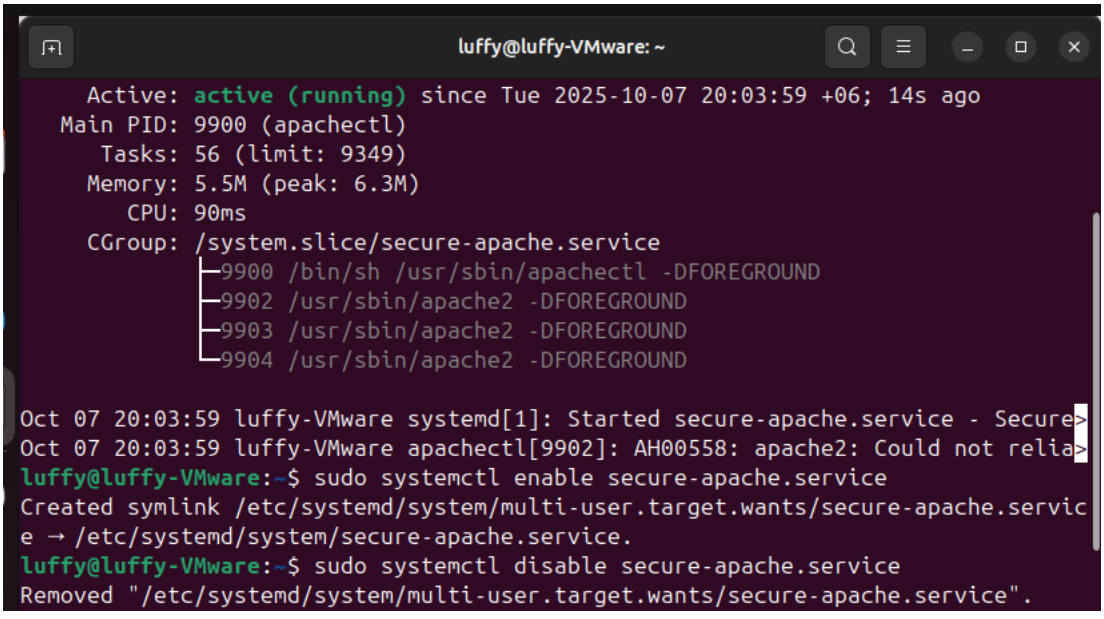
Enabling the service to auto-start

The service starts automatically when the system restarts.

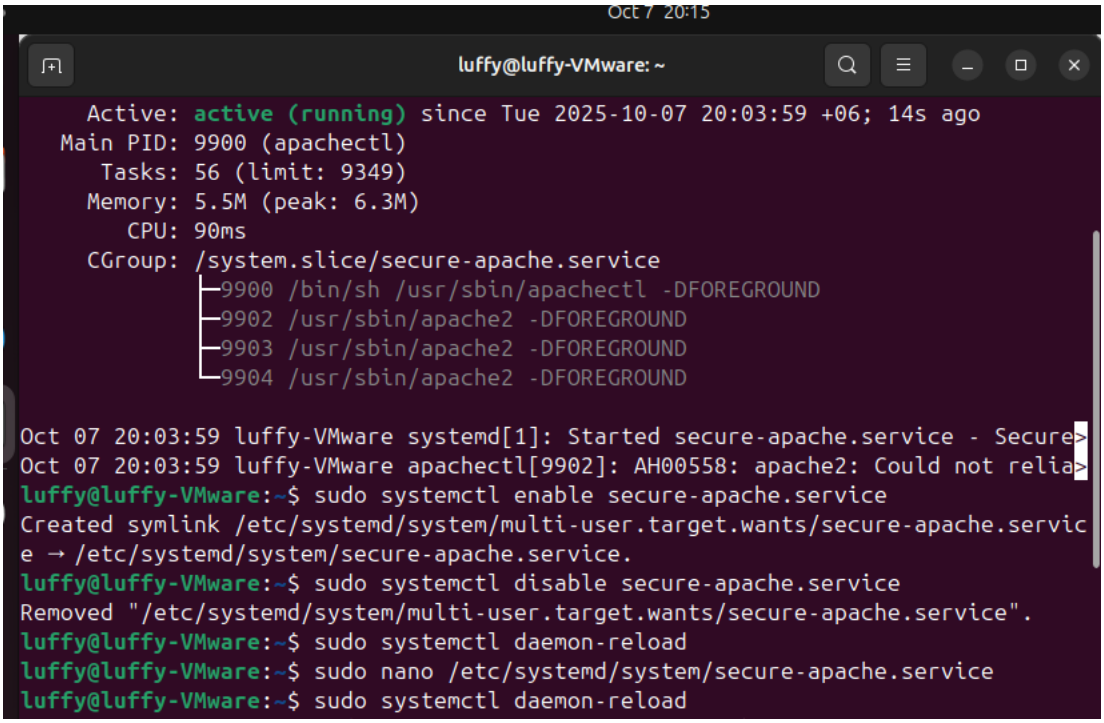


Stopping service auto-start

It no longer auto-starts upon system startup.



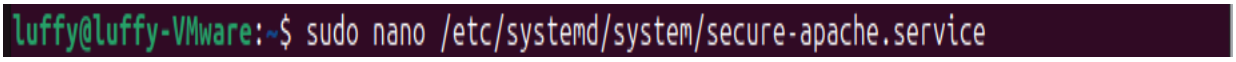
Reloading (resulting from changing the service file)



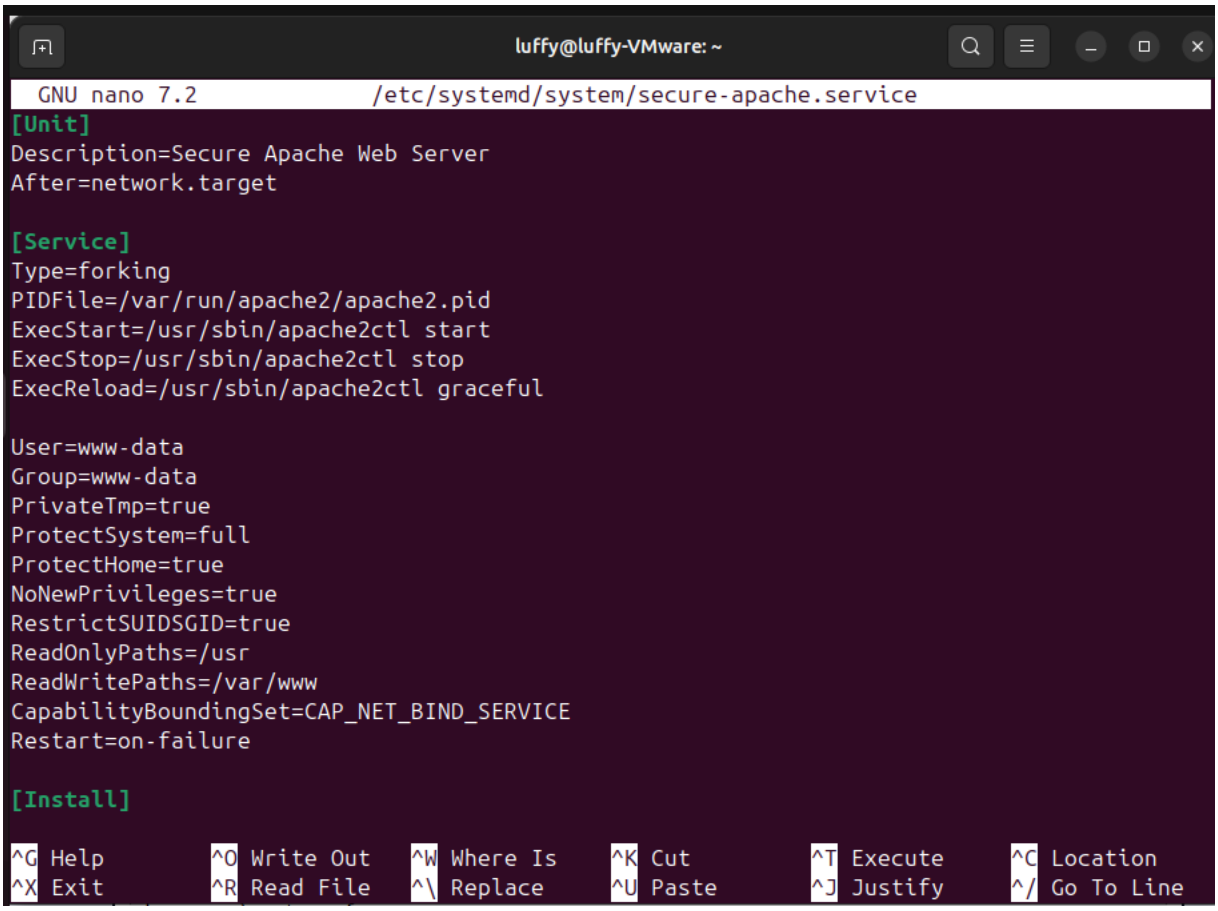
Now systemd services can be fully managed - start, stop, restart, enable, disable, and status everything.

Security Setup:

Service file was opened**.**



The following secure service configuration has been written and saved.

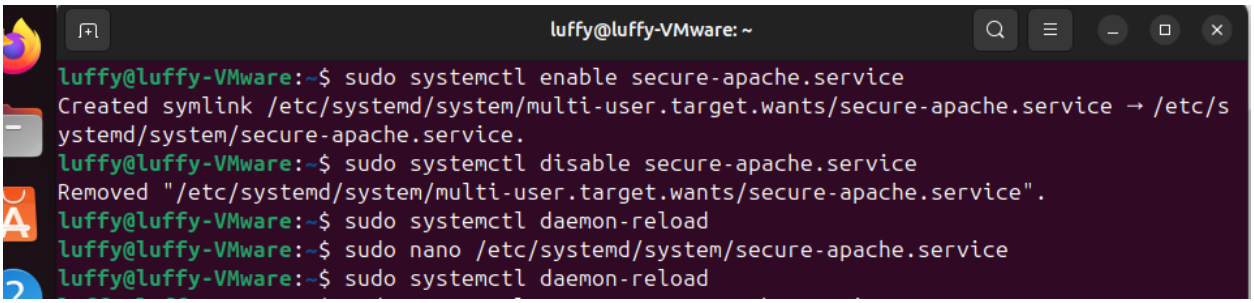


These directives are mainly used to make a service or daemon run more securely. The service does not run as the root user, but rather as the limited www-data user, so that it does not have full control over the system. PrivateTmp=true means that a separate temporary /tmp directory is created for the service, which is not shared with other services. ProtectSystem=full makes the system's core files read-only, so that the service cannot modify any important system files. The ProtectHome=true setting blocks the service's access to the /home directory, so that personal files are safe. NoNewPrivileges=true ensures that any child processes created by the service do not gain any new privileges or additional permissions. The RestrictSUIDSGID=true option restricts the use of setuid and setgid files, reducing the risk of privilege escalation.

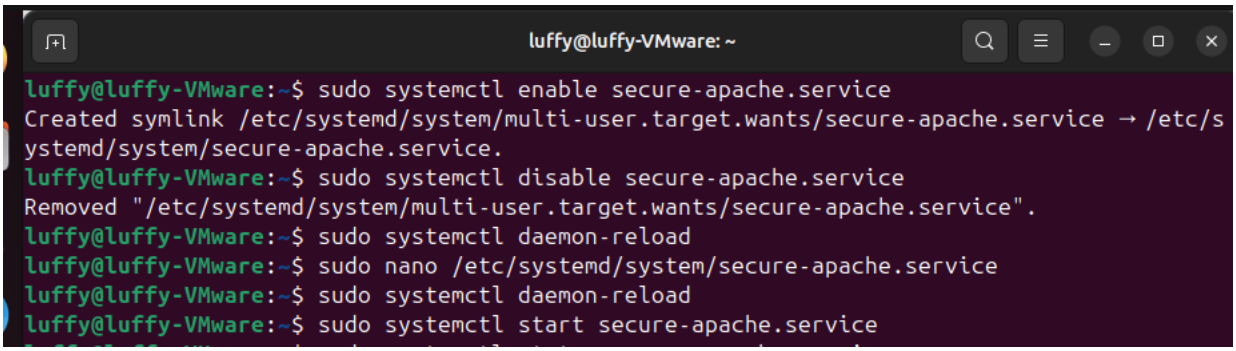
CapabilityBoundingSet=CAP_NET_BIND_SERVICE allows the service to use only network ports 80 and 443. This limits the limited network access required by the web service. The

Restart=on-failure setting automatically restarts the service if it crashes or stops for some reason [9], [10].

Reloading Systemd



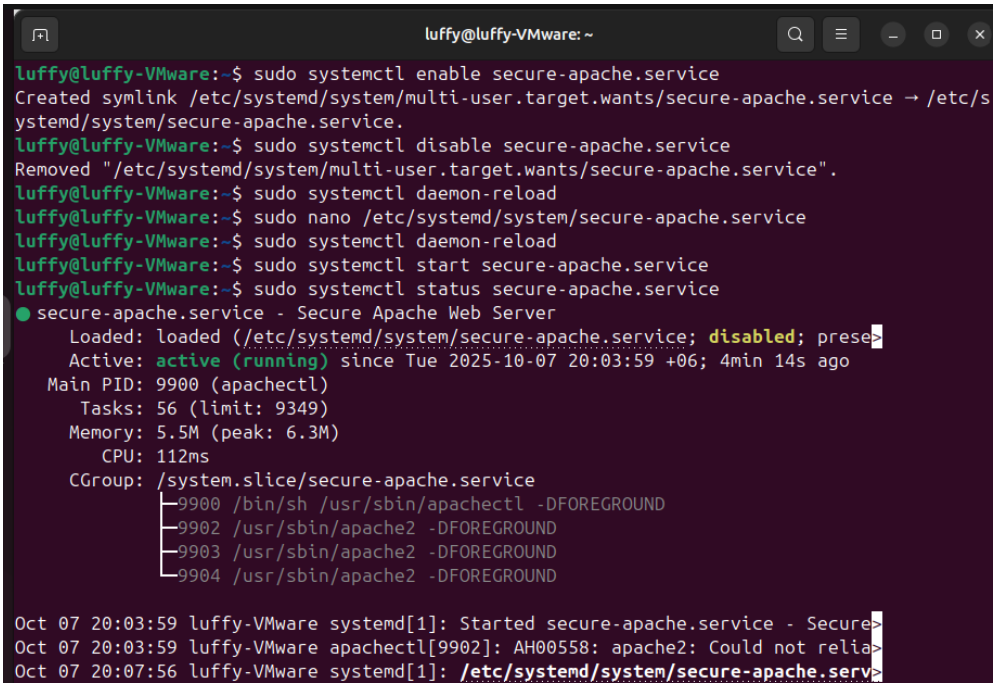
Running the service



Checking the service status

Shows the output:

Active: active (running)



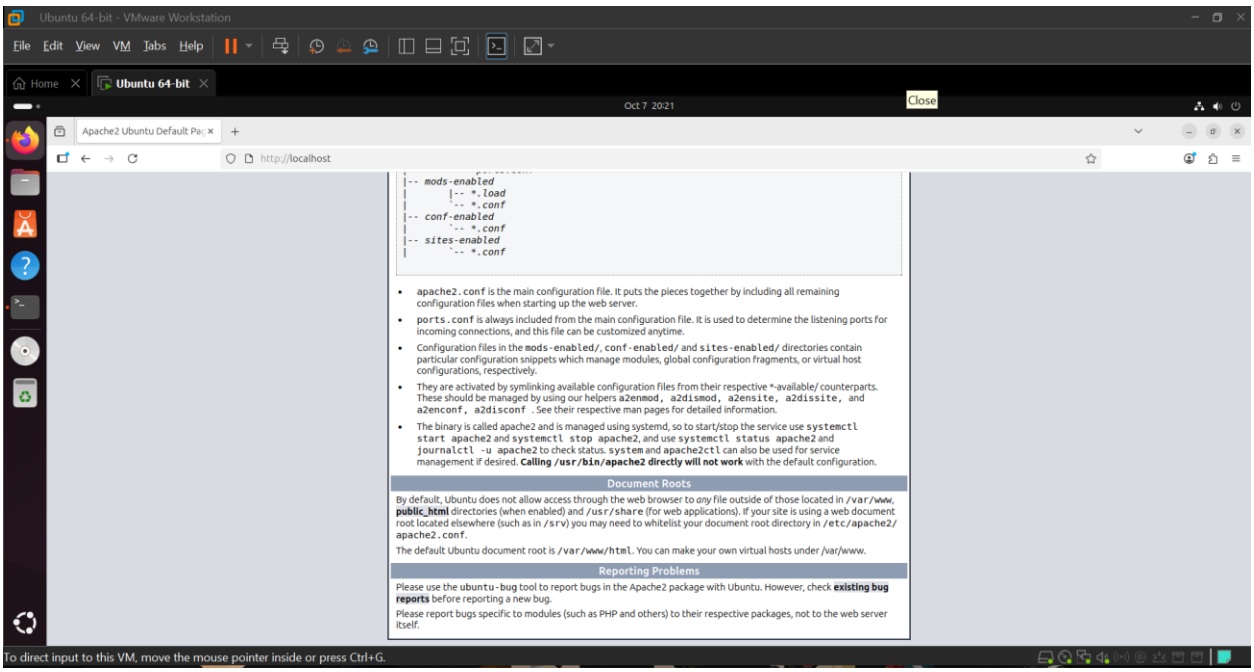
Verifying Apache

Opening the browser and type:

http://localhost

This shows the Apache2 Default Page, which indicates that the service is running properly.


Ubuntu 64-bit - VMware Workstation
File Edit View VM Tabs Help
Home
Ubuntu 64-bit
Oct 7 20:20
Close
Apache2 Ubuntu Default Pa
http://localhost
Apache2 Default Page
Ubuntu
It works!
This is the default welcome page used to test the correct operation of the Apache2 server after installation on Ubuntu systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should replace this file (located at /var/www/html/index.html) before continuing to operate your HTTP server.
If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.
Configuration Overview
Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is fully documented in /usr/share/doc/apache2/README.Debian.gz. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the manual if the apache2-doc package was installed on this server.
The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:
/etc/apache2/
|-- apache2.conf
| `-- ports.conf
|-- mods-enabled
| |-- *.load
| `-- *.conf
|-- conf-enabled
| `-- *.conf
|-- sites-enabled
| `-- *.conf
apache2.conf is the main configuration file. It puts the pieces together by including all remaining configuration files when starting up the web server.
To direct input to this VM, move the mouse pointer inside or press Ctrl+G.


Ubuntu 64-bit - VMware Workstation
File Edit View VM Tabs Help
Home
Ubuntu 64-bit
Oct 7 20:21
Close
Apache2 Ubuntu Default Pa
http://localhost
|-- mods-enabled
| |-- *.load
| `-- *.conf
|-- conf-enabled
| `-- *.conf
|-- sites-enabled
| `-- *.conf
apache2.conf is the main configuration file. It puts the pieces together by including all remaining configuration files when starting up the web server.
ports.conf is always included from the main configuration file. It is used to determine the listening ports for incoming connections, and this file can be customized anytime.
Configuration files in the mods-enabled/, conf-enabled/ and sites-enabled/ directories contain particular configuration snippets which manage modules, global configuration fragments, or virtual host configurations, respectively.
They are activated by symlinking available configuration files from their respective *-available/ counterparts. These should be managed by using our helpers a2enmod, a2dismod, a2ensite, a2dissite, and a2enconf, a2disconf . See their respective man pages for detailed information.
The binary is called apache2 and is managed using systemd, so to start/stop the service use systemctl start apache2 and systemctl stop apache2, and use systemctl status apache2 and journalctl -u apache2 to check status. system and apache2ctl can also be used for service management if desired. Calling /usr/bin/apache2 directly will not work with the default configuration.
Document Roots
By default, Ubuntu does not allow access through the web browser to any file outside of those located in /var/www, public_html directories (when enabled) and /usr/share (for web applications). If your site is using a web document root located elsewhere (such as in /srv) you may need to whitelist your document root directory in /etc/apache2/apache2.conf.
The default Ubuntu document root is /var/www/html. You can make your own virtual hosts under /var/www.
Reporting Problems
Please use the ubuntu-bug tool to report bugs in the Apache2 package with Ubuntu. However, check existing bug reports before reporting a new bug.
Please report bugs specific to modules (such as PHP and others) to their respective packages, not to the web server itself.
To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

Task 4: Crontab

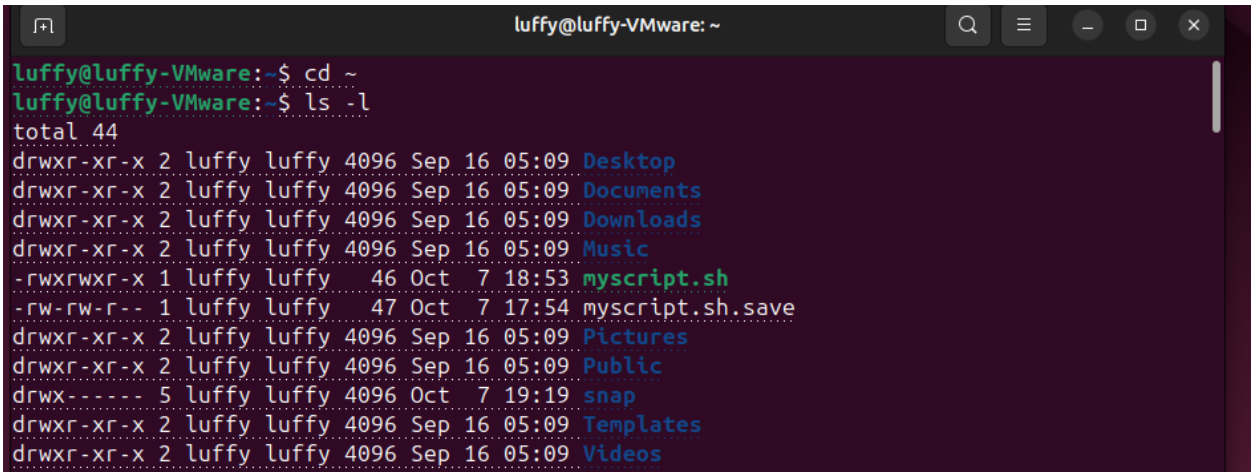
Set the shell script that you have written in Task 2 to run in a crontab, configure the crontab to run every 30 minute on Wednesday

Solution:

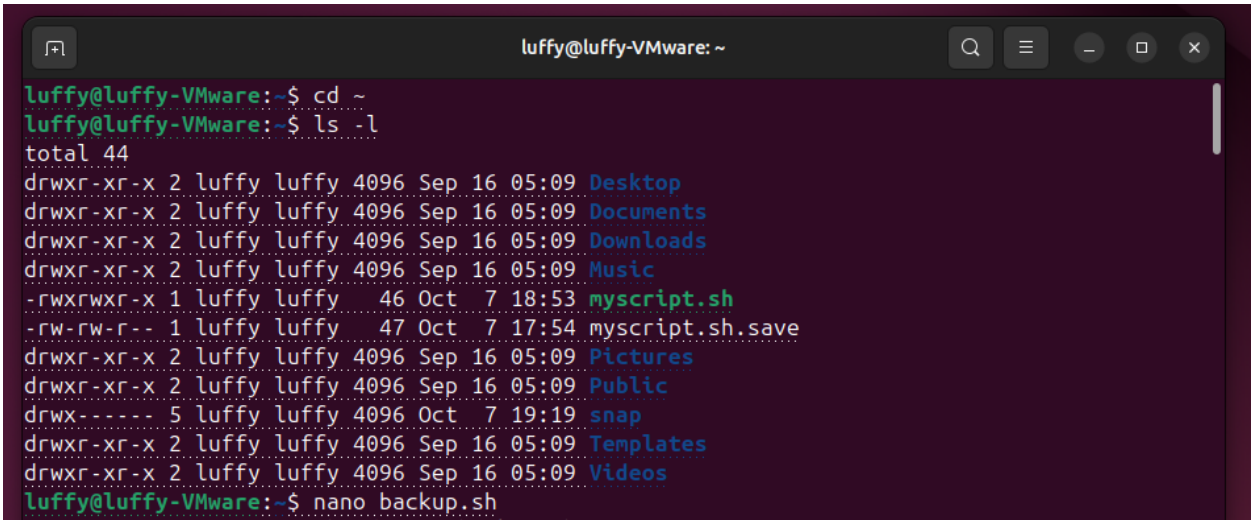
I opened Ubuntu from VMware Workstation from Windows and open Terminal.

Check if Shell Script exists or not

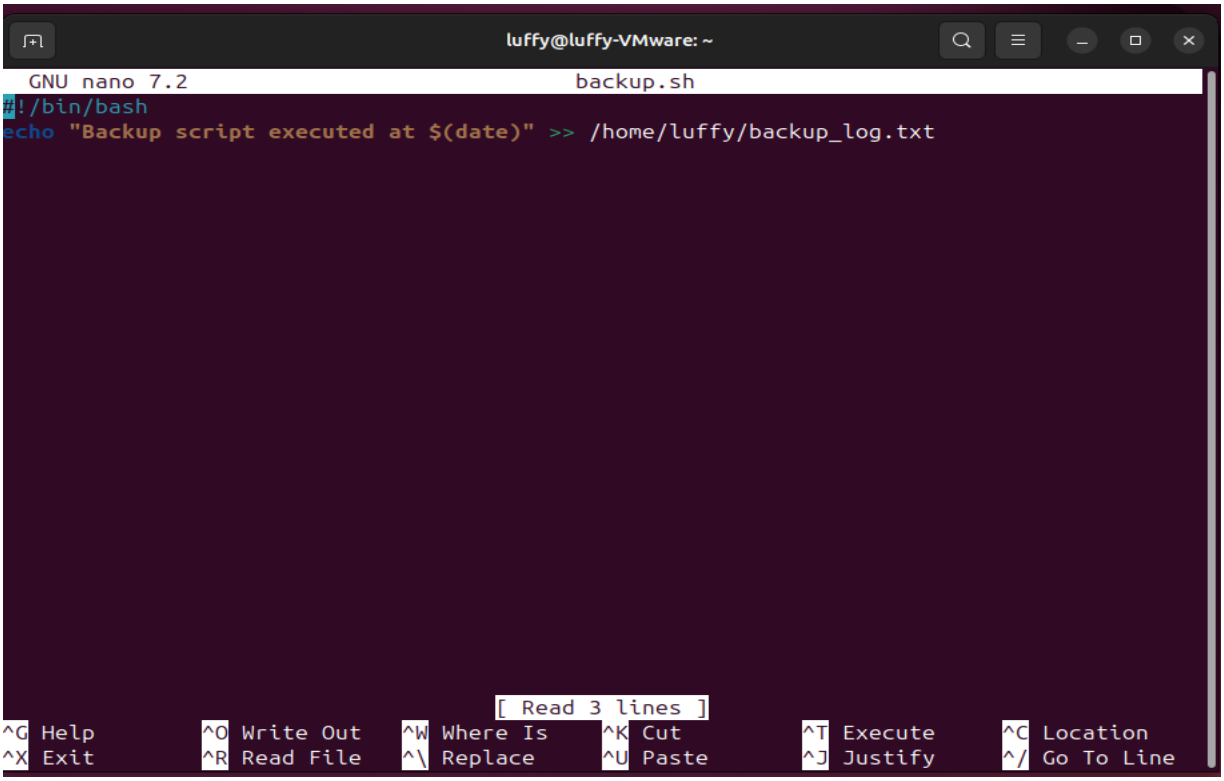
The script has been created earlier which is named backup.sh and it is placed in /home/luffy/. Then I typed ls -l in Terminal.



The file named backup.sh was not found. Since the file did not exist, a new file was created:



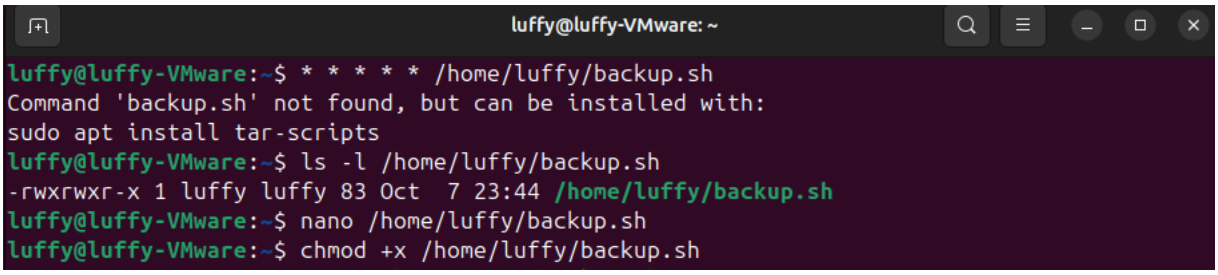
Then the following small script was added to the editor [11].



For saving, I pressed Ctrl + O and then pressed Enter, and to exit, pressed Ctrl + X.

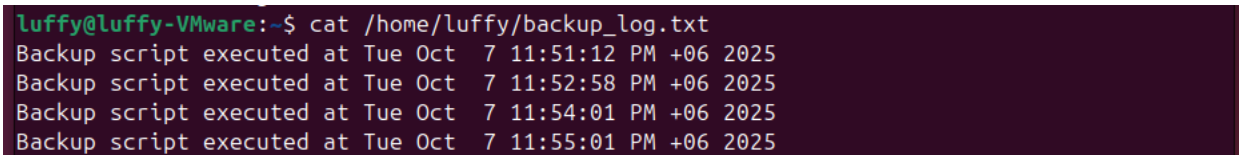
Execution permission for the script

Execution permission is given by typing in the terminal:



The chmod command allows the script to be executed [11].

The following command is typed in the terminal:



Opening crontab

I opened crontab by typing in the terminal:



It shows installing new crontab, which means that crontab has been successfully opened.

Adding a new Cron Job line

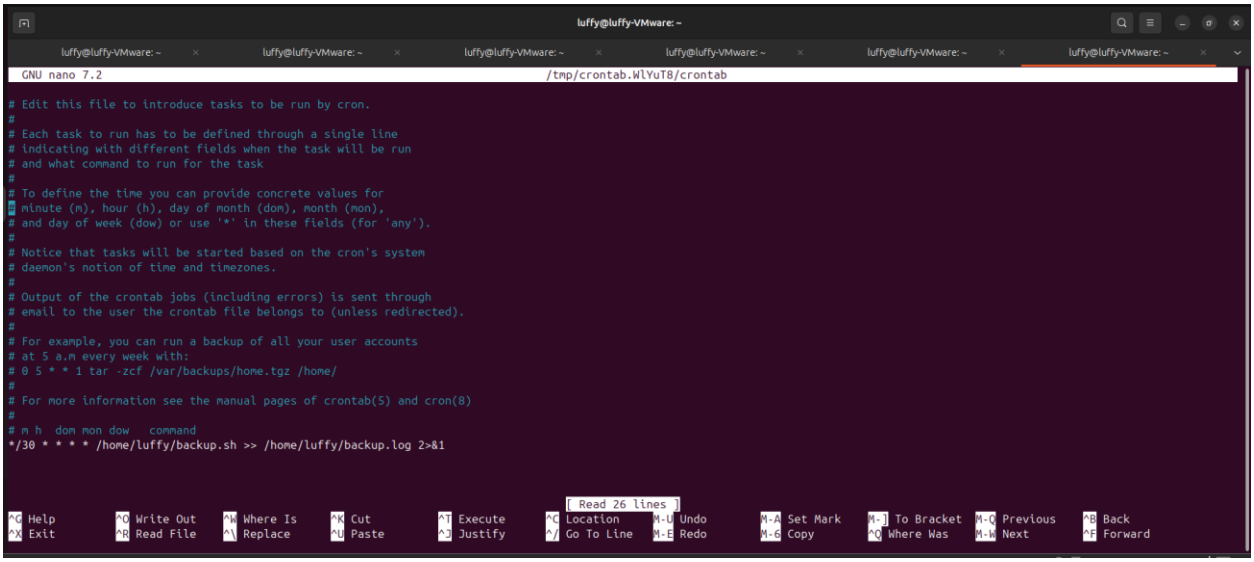
When the Crontab file is opened, the arrow key down on the keyboard is pressed to go down,

Then the following line is typed and added (changed according to my path):

*/30 * * * * /home/luffy/backup.sh >> /home/luffy/backup.log 2>&1

This cron expression instructs the script to run according to a specific schedule. Here, the */30 * * * * part means that this job runs every 30 minutes. The script is automatically started at 00 and 30

minutes of every hour. /home/luffy/backup.sh is the full path of the backup script that is run. >> /home/luffy/backup.log indicates that any output generated when the script is run will be appended to the backup.log file. The previous lines will not be deleted. The 2>&1 part ensures that if any errors or error messages occur while running the script, they are also saved in the same log file. As a result, the backup.log file keeps a record of all the script's activities and possible errors [11], [12].



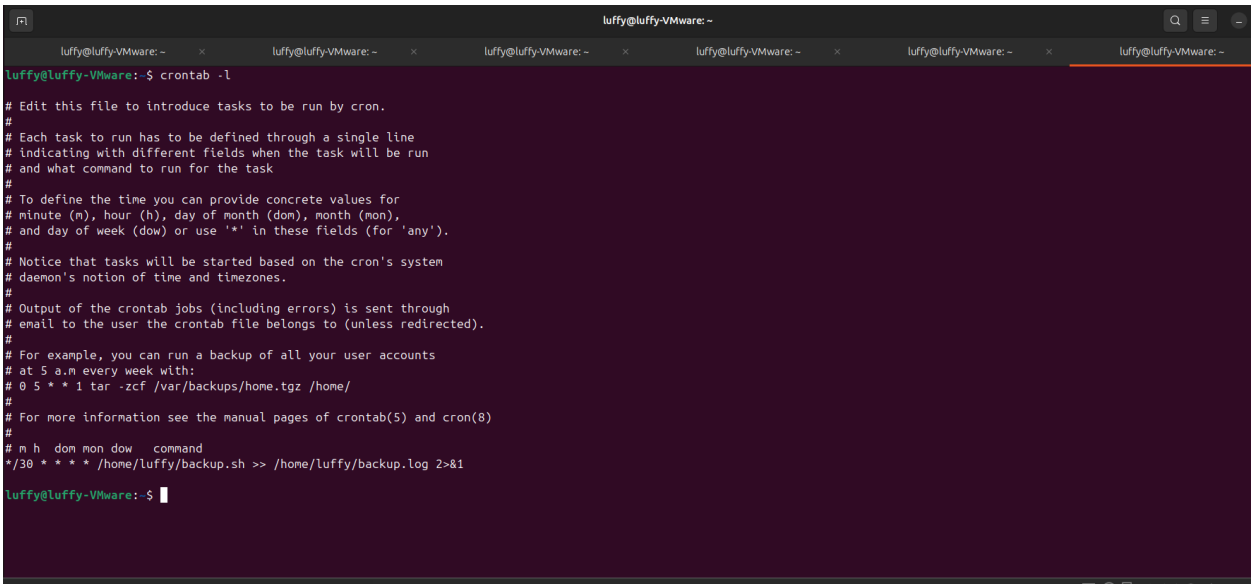
Saving the crontab file

In Nano, press Ctrl + O, press Enter, save, and press Ctrl + X to exit the editor [12], [13].

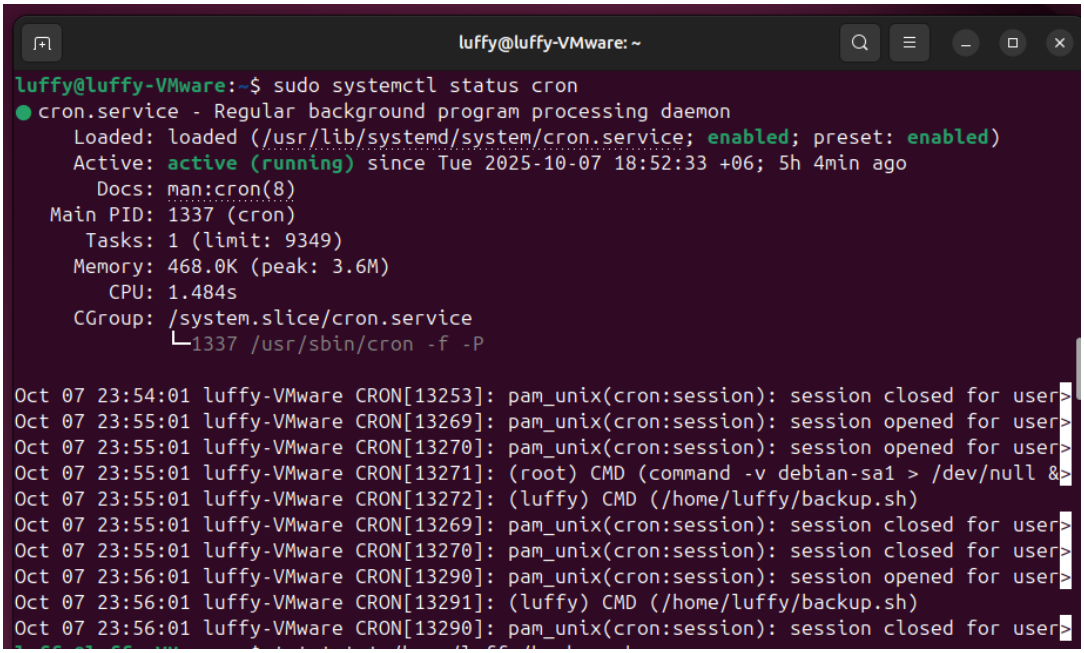
The terminal shows: installing new crontab

Verifying whether Cron is working

This shows the previously added line:



To check if the Cron service is running:



Tells systemd to start the cron service automatically at boot time.

```
luffy@luffy-VMware:~$ sudo systemctl enable cron
Synchronizing state of cron.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable cron
```

This command tells systemd that the cron service will start automatically at boot time. cron will continue to run after reboot [13].

Running the backup script

```
luffy@luffy-VMware:~$ bash /home/luffy/backup.sh
```

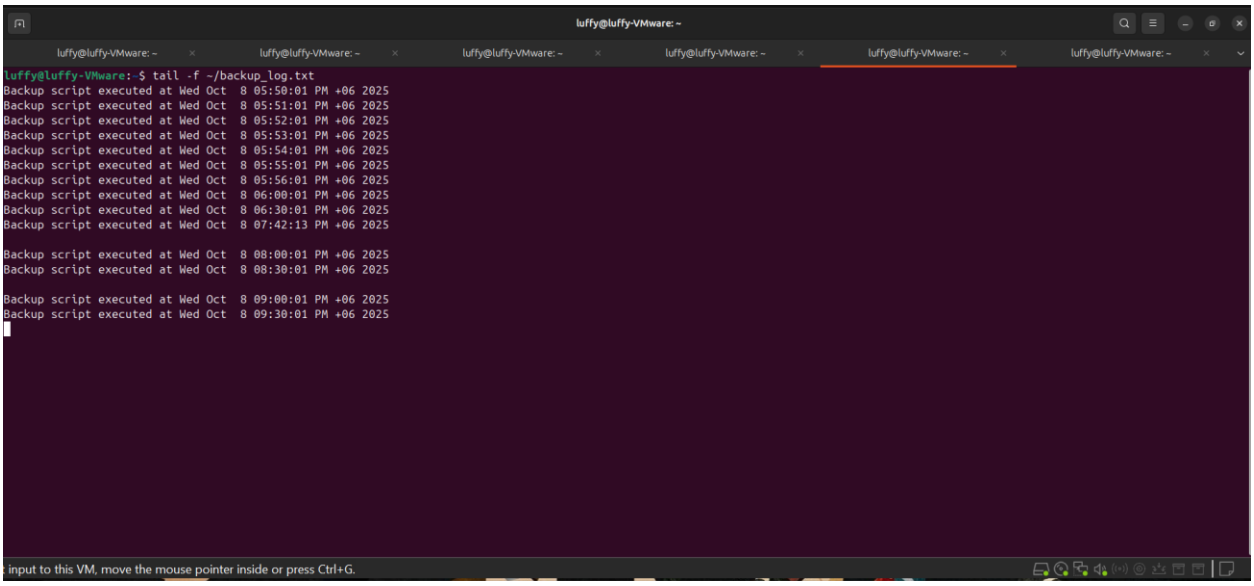
This command runs the backup script once at that moment.

Showing the last lines of the backup.log file and bringing it to the terminal with new lines when the file is updated:

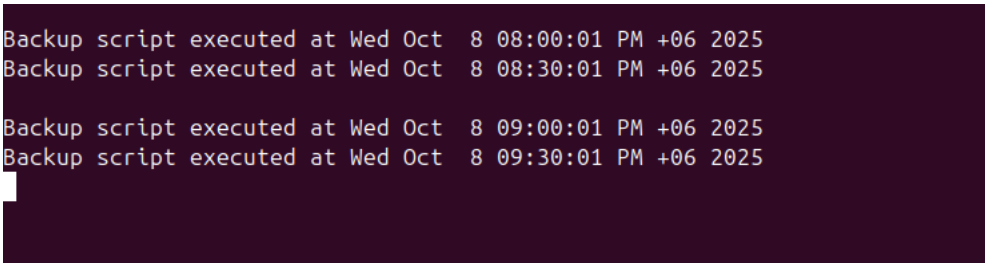
```
luffy@luffy-VMware:~$ tail -f ~/backup.log | awk '{split($6,t,":"); if(t[2]==0 || t[2]==30) print}'
```

This command is mainly used for live log monitoring so that the output of the backup script can be seen in real time after it is run. The tail -f ~/backup.log part shows the last few lines of the backup.log file and automatically displays the newly added lines in the terminal when the file is updated. As a result, a kind of live monitoring or real-time log is seen. Then the pipe (|) symbol is used to send the continuous stream of lines coming from tail -f to the next command awk. The data of each line is processed in the awk '{split($6,t,""); if(t[2]==0 || t[2]==30) print}' part. Here, the split($6,t,"":") instruction divides the 6th part of the line (where the time is in HH:MM:SS format) by : and stores it in an array named t. t[2] of that array is the minute value. The Task says that only those lines will be printed if the minute value is 00 or 30. That is, using this entire command together, only the log entries created every 30 minutes in the backup.log file can be viewed in real time in the terminal [13], [14].

Checking results



The new line has been added after 6.00 pm. Here, crontab is running every 30 minutes from 8.00 pm to 9.30 pm. That means Crontab is working.



This means that crontab is working properly in the system and is running continuously in the background.

References

[1] “Manage Systemd Services with systemctl on Linux,” *DigitalOcean*, accessed. Available: https://www.digitalocean.com/community/tutorials/how-to-use-systemctl-to-manage-systemd-services-and-units

[2] “Linux Runlevels Explained,” *Liquid Web*, accessed. Available: https://www.liquidweb.com/blog/linux-runlevels-explained/

[3] “systemctl - Control the systemd system and service manager,” *freedesktop.org*, accessed. Available: https://www.freedesktop.org/software/systemd/man/systemctl.html

[4] “Introduction to systemd Basics,” *SUSE Documentation*, accessed. Available: https://documentation.suse.com/smart/systems-management/html/systemd-basics/index.html

[5] “Init vs Systemd,” *Cycle.io*, accessed. Available: https://cycle.io/learn/init-vs-systemd

[6] “systemd.service — Service unit configuration,” *Freedesktop.org*, 2024. Available: https://www.freedesktop.org/software/systemd/man/systemd.service.html

[7] “How to Create a systemd Service in Linux,” *DigitalOcean Tutorials*, Apr. 2023. Available: https://www.digitalocean.com/community/tutorials/how-to-create-a-systemd-service-in-linux

[8] “Bash Scripting Tutorial,” *GNU Bash Reference Manual*, Free Software Foundation, 2024. Available: https://www.gnu.org/software/bash/manual/

[9] J. Siegel, “systemd: Service Management Made Easy,” *Red Hat Documentation*, 2023. Available: https://access.redhat.com/documentation

[10] Ubuntu Documentation, “Systemd overview,” *Ubuntu Server Guide*, Canonical Ltd., 2024. Available: https://ubuntu.com/server/docs

[11] “CronHowto,” *Ubuntu Community Help Wiki*, Canonical Ltd., 2024. Available: https://help.ubuntu.com/community/CronHowto

[12] “Crontab Guru - The quick and simple editor for cron schedule expressions,” *Crontab.guru*, 2025. [Online]. Available: https://crontab.guru/

[13] “Systemd and Cron jobs in Ubuntu,” *DigitalOcean Tutorials*, 2024. Available: https://www.digitalocean.com/community/tutorials/how-to-schedule-tasks-with-cron

[14] “GNU nano Manual,” *GNU Project*, 2024. Available: https://www.nano-editor.org/dist/latest/nano.html